# TA Assignment RahulBhujade

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
[ ] #111_RahulBhujade
    # The code below works to create a separate dataframe that one can filter down to with the polarity sco
    #First install the required packages using following commands-
    #pip install -U textblob
    #python -m textblob.download_corpora

    import pandas as pd            #Pandas
    from textblob import TextBlob #For Sentiment Anlysis
    from itertools import islice #To iterate and slice the tuple read from the file
```

```
#df_survey_data = pd.read_csv(r'C:\Users\Chaudhary\Desktop\LearninTxtAnalytics\OnePluseReviews1.csv')
#df_survey_data
from google.colab import files
uploaded = files.upload()
```

Choose Files OnePluseReviews1.csv

- **OnePluseReviews1.csv**(text/csv) - 452397 bytes, last modified: 9/21/2023 - 100% done

```
Saving OnePluseReviews1.csv to OnePluseReviews1.csv
```

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
[ ] #111_RahulBhujade
    import pandas as pd
    import io
    df2 = pd.read_csv(io.BytesIO(uploaded['OnePluseReviews1.csv']))
    #df2 = pd.read_csv(r'OnePluseReviewsComments.csv') #for Anaconda
```

```
[ ] COLS = ['text', 'sentiment','subjectivity','polarity'] #create dataframe COLS
```

```
[ ] df = pd.DataFrame(columns=COLS)
```

```
# find subjectivity and polarity
for index, row in islice(df2.iterrows(), 0, None):    #islice(iterable, start, stop, step)
    new_entry = [] # list
    text = (index, row['Comment'])    #Iterating rows and storing index and comment value
    text_str = str(text)
    text_lower = text_str.lower() #convert to lower case
    blob = TextBlob(text_lower)  #TextBlob Library aims to provide access to common text-processing ope
    #You can treat TextBlob objects as if they were Python strings that learned how to do Natural Langu
    sentiment = blob.sentiment
    #The sentiment property returns a namedtuple of the form Sentiment(polarity, subjectivity).
    #Subjectivity - In natural language, subjectivity refers to expression of opinions, evaluations, fe
```

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
#The sentiment property returns a namedtuple of the form Sentiment(polarity, subjectivity).
#Subjectivity - In natural language, subjectivity refers to expression of opinions, evaluations, fe
#The subjectivity is a float within the range [0.0, 1.0] where 0.0 is very objective and 1.0 is ver
#Polarity - whether the expressed opinion in a document, a sentence or an entity feature/aspect is
# Polarity is based on or influenced by personal feelings, tastes, or opinions. The polarity score
polarity = sentiment.polarity
subjectivity = sentiment.subjectivity
#if the polarity is >0, it is considered positive, <0 -is considered negative and ==0 is considered

# make a new entry with Comment in lower case, sentiment, subjectivity, and polarity
new_entry += [text_lower,sentiment,subjectivity,polarity] # =+ is a compound operator that adds to

single_survey_sentimet_df = pd.DataFrame([new_entry], columns=COLS)
df = df.append(single_survey_sentimet_df, ignore_index=True)
#append() function is used to append rows of other dataframe to the end of the given dataframe, returni
#Columns not in the original dataframes are added as new columns and the new cells are populated with N
#ignore_index : If True, do not use the index labels.
```

```
  df = df.append(single_survey_sentimet_df, ignore_index=True)
<ipython-input-6-bbda6f8e0a85>:23: FutureWarning: The frame.append method is deprecated and will be re
  df = df.append(single_survey_sentimet_df, ignore_index=True)
<ipython-input-6-bbda6f8e0a85>:23: FutureWarning: The frame.append method is deprecated and will be re
  df = df.append(single_survey_sentimet_df, ignore_index=True)
<ipython-input-6-bbda6f8e0a85>:23: FutureWarning: The frame.append method is deprecated and will be re
```

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
#111_RahulBhujade
#Convert dataframe to .csv file 'SentimentPolarity_Values.csv'
df.to_csv('SentimentPolarity_Values.csv', mode='w', columns=COLS, index=False, encoding="utf-8")
```

```
[ ] df.head()     # Returns the first 5 rows of the dataframe.
#To override the default, you may insert a value between the parenthesis to change the number of rows r
```

| | text | sentiment | subjectivity | polarity |
|---|---|---|---|---|
| 0 | (0, "the same thing happening in the new repla... | (-0.07727272727272728, 0.3215909090909091) | 0.321591 | -0.077273 |
| 1 | (1, "everything is best except the thing that ... | (0.4166666666666667, 0.3666666666666667) | 0.366667 | 0.416667 |
| 2 | (2, 'do watch the video. m very disappointed w... | (-0.9750000000000001, 0.9750000000000001) | 0.975000 | -0.975000 |
| 3 | (3, 'awesome device, and hats off to amazon fo... | (0.3291666666666667, 0.6645833333333333) | 0.664583 | 0.329167 |
| 4 | (4, 'worth every buck.finally one plus has inv... | (0.5800000000000001, 0.45999999999999996) | 0.460000 | 0.580000 |

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
#111_RahulBhujade
df.describe() # describe the dataframe
```

| | subjectivity | polarity |
|---|---|---|
| count | 1400.000000 | 1400.000000 |
| mean | 0.541302 | 0.299104 |
| std | 0.222256 | 0.333870 |
| min | 0.000000 | -1.000000 |
| 25% | 0.432857 | 0.067417 |
| 50% | 0.546375 | 0.299872 |
| 75% | 0.660000 | 0.500000 |
| max | 1.000000 | 1.000000 |

```
[ ] dffilter = df.loc[(df.loc[:, df.dtypes != object] != 0).any(1)]
    #df.loc - attribute to access a particular cell in the given Dataframe using the index and column label
```

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
#111_RahulBhujade
dffilter = df.loc[(df.loc[:, df.dtypes != object] != 0).any(1)]
#df.loc - attribute to access a particular cell in the given Dataframe using the index and column label
#loc() function is used to access a group of rows and columns by label(s) or a boolean array.
#Returns a cross-section (row(s) or column(s)) from the Series/DataFrame. Access group of values using
```

```
<ipython-input-10-44a80b2d8881>:1: FutureWarning: In a future version of pandas all arguments of DataFra
  dffilter = df.loc[(df.loc[:, df.dtypes != object] != 0).any(1)]
```

```
dffilter.describe()
```

| | subjectivity | polarity |
|---|---|---|
| count | 1328.000000 | 1328.000000 |
| mean | 0.570650 | 0.315320 |
| std | 0.187931 | 0.335261 |
| min | 0.000000 | -1.000000 |
| 25% | 0.453010 | 0.123037 |

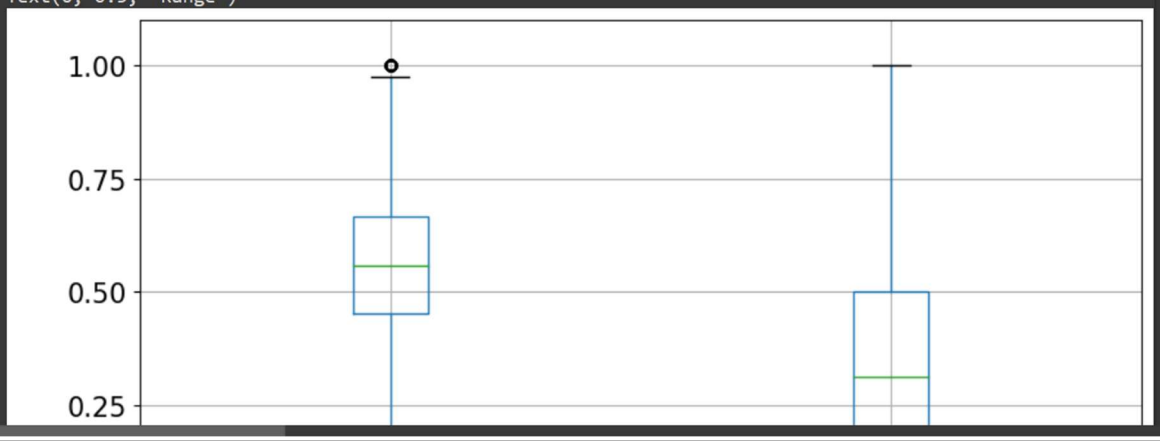
Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
import matplotlib.pyplot as plt
import seaborn as sns
#Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface
boxplot = dffilter.boxplot(column=['subjectivity','polarity'],
                     fontsize = 15,grid = True, vert=True,figsize=(10,10,))
plt.ylabel('Range') # label y=axis as 'Range'
```

```
Text(0, 0.5, 'Range')
```



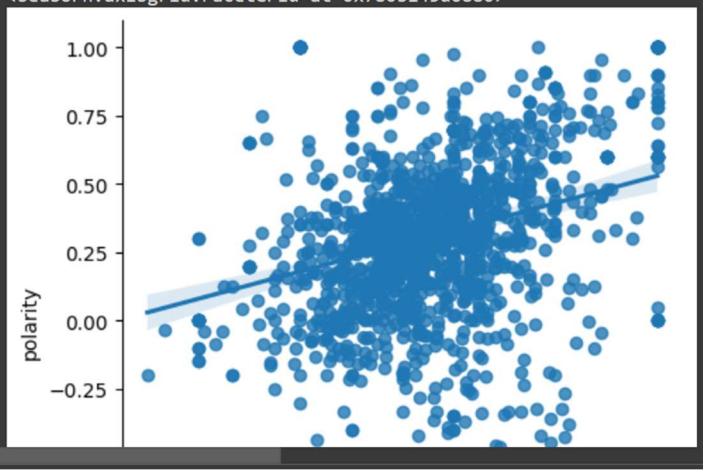

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
#scatter for dffilter
#111_RahulBhujade
sns.lmplot(x='subjectivity',y='polarity',data=dffilter,fit_reg=True,scatter=True, palette="mute")
# lmplot is a seaborn function intended as a convenient interface to fit regression models across condi
```

```
<seaborn.axisgrid.FacetGrid at 0x780b145d0880>
```



Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```python
# calculate the covariance between two variables
#111_RahulBhujade
from numpy.random import randn ## Return a sample (or samples) from the “standard normal” distribution.
from numpy.random import seed # seed of last ramdom number generated - NumPy random seed is simply a fu
from numpy import cov #Estimate a covariance matrix, given data and weights - https://numpy.org/doc/sta
from scipy.stats import pearsonr #Calculate a Pearson correlation coefficient and the p-value for testi

# prepare data
data1 = dffilter['subjectivity']
data2 = data1 + dffilter['polarity']
# calculate covariance matrix
covariance = cov(data1, data2)
print("Covariance")
print(covariance)
#print(covariance) matrix of subjectivity vs polarity. +ve values indicate same direction
# Row1 Column1  shows Variance of data1,  Row1 Column 2   or   Row2 Column1 shows COV of data1 wrt data
corr, _ = pearsonr(data1, data2)
print('Pearsons correlation: %.5f' % corr) # float value upto 5 decimal points
```

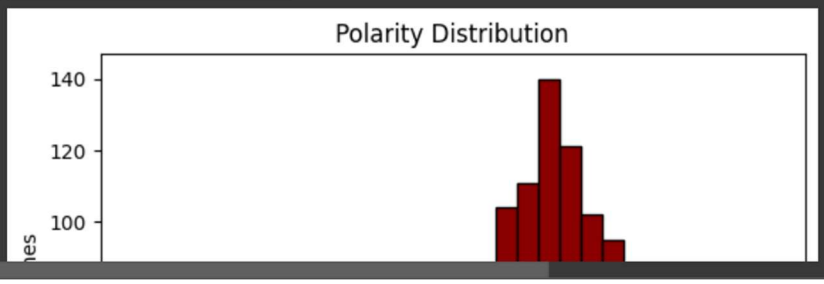
```
Covariance
[[0.03531801 0.05299256]
 [0.05299256 0.18306694]]
Pearsons correlation: 0.65904
```

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```python
#Polarity Distribution for dffilter #111_RahulBhujade
# plot histogram for polarity keeping default as density=False
plt.hist(dffilter['polarity'], color = 'darkred', edgecolor = 'black', density=False,
         bins = int(30))
# If density=True, the first element of the return tuple will be the counts normalized to form a probab

plt.title('Polarity Distribution')
plt.xlabel("Polarity") # label x-axis as Polarity
plt.ylabel("Number of Times") # label y-axis as Number of Times

from pylab import rcParams
# rcParams changes the default parameters -> Each time Matplotlib loads, it defines a runtime configura
rcParams['figure.figsize'] = 5,5
```



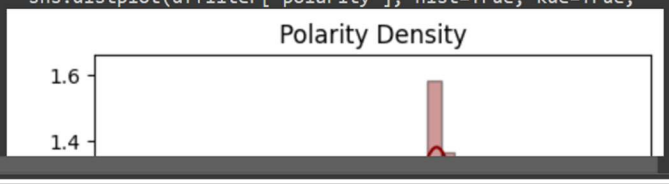
Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
sns.distplot(dffilter['polarity'], hist=True, kde=True,
             bins=int(30), color = 'darkred',
             hist_kws={'edgecolor':'black'},axlabel ='Polarity')
plt.title('Polarity Density')

from pylab import rcParams
rcParams['figure.figsize'] = 10,15
```

```
<ipython-input-16-c404a6f1765f>:3: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(dffilter['polarity'], hist=True, kde=True,
```




Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
#111_RahulBhujade
import nltk
nltk.download('all')
```

```
[nltk_data] Downloading collection 'all'
[nltk_data]    |
[nltk_data]    | Downloading package abc to /root/nltk_data...
[nltk_data]    |   Unzipping corpora/abc.zip.
[nltk_data]    | Downloading package alpino to /root/nltk_data...
[nltk_data]    |   Unzipping corpora/alpino.zip.
[nltk_data]    | Downloading package averaged_perceptron_tagger to
[nltk_data]    |     /root/nltk_data...
[nltk_data]    |   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data]    | Downloading package averaged_perceptron_tagger_ru to
[nltk_data]    |     /root/nltk_data...
[nltk_data]    |   Unzipping
[nltk_data]    |       taggers/averaged_perceptron_tagger_ru.zip.
[nltk_data]    | Downloading package basque_grammars to
[nltk_data]    |     /root/nltk_data...
[nltk_data]    |   Unzipping grammars/basque_grammars.zip.
[nltk_data]    | Downloading package bcp47 to /root/nltk_data...
[nltk_data]    | Downloading package biocreative_ppi to
[nltk_data]    |     /root/nltk_data...
[nltk_data]    |   Unzipping corpora/biocreative_ppi.zip.
```

✓ 0s completed at 10:34 AM

Help Cannot save changes

Share

+ Code + Text Copy to Drive

Reconnect

```
[ ] stopwords = nltk.corpus.stopwords.words('english') #import stopwords from the corpus of english languag
```

```
#111_RahulBhujade# REmove stopwords: pandas.Series.str.replace
RE_stopwords = r'\b(?:{})\b'.format('|'.join(stopwords))
# refer to regex: https://docs.python.org/3.1/library/re.html

# exclude stopwords using pandas.Series.str.replace
words = (df.text
           .str.lower()           # convert the sentiments to lower case
           .replace([r'\|',r'\&',r'\-',r'\.',r'\,',r'\''', RE_stopwords], [' ', '','','','','','''], rege
            # str.replace to perform string or regex substitution on series of string data as above
                     # replace the two character string having '\' and '|' with ' '
                          # replace the two character string having '\' and '&' with ''
                                # replace the two chracter string having '\' and '-' with ''
                                      # replace the two chracter string having '\' and '.' with ''
                                             # replace the two chracter string having '\' and ',' with
                                                 # replace the two chracter string having '\' and ''' w
                                                         # replace RE_stopwords with ''
           .str.cat(sep=' ') #concatenate strings in pandas with separator put in between
           .split() # string.split() returns a list of strings after breaking the given string by the s
)
```

Help Cannot save changes

Share

+ Code + Text Copy to Drive

Reconnect

```
#111_RahulBhujade# Word Frequency of words from above
from collections import Counter # Python Counter is a container that will hold the count of each of the
                                # Counter is a sub-class that is used to count hashable objects. It imp
# generate DF out of Counter
rslt = pd.DataFrame(Counter(words).most_common(10), # 10 most common words
                    columns=['Word', 'Frequency']).set_index('Word')
rslt
```

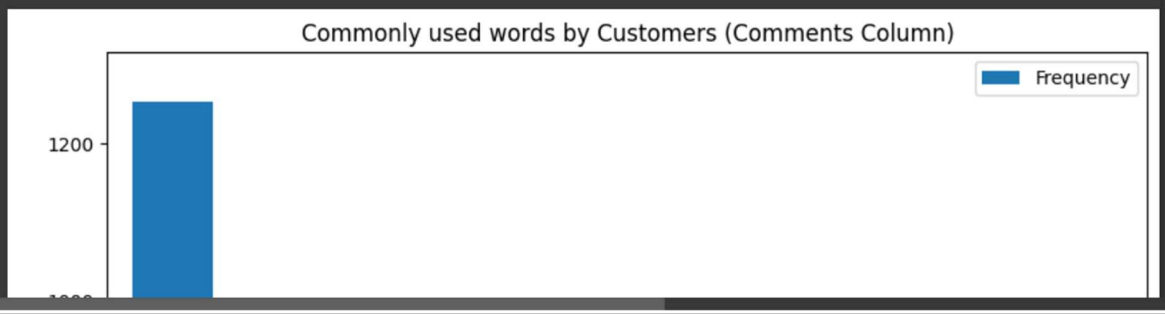
| Word | Frequency |
|---|---|
| phone | 1253 |
| camera | 781 |
| good | 600 |
| one | 579 |
| battery | 475 |
| oneplus | 416 |
| display | 405 |

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```python
#111_RahulBhujade
rslt_wordcloud = pd.DataFrame(Counter(words).most_common(100),
                    columns=['Word', 'Frequency'])
#BAR CHART of 10 commonly used words by customers
rslt.plot.bar(rot=40, figsize=(10,10), width=0.8,colormap='tab10')
plt.title("Commonly used words by Customers (Comments Column)")
plt.ylabel("Count")

from pylab import rcParams
rcParams['figure.figsize'] = 10,15
```




s Help Cannot save changes

Share

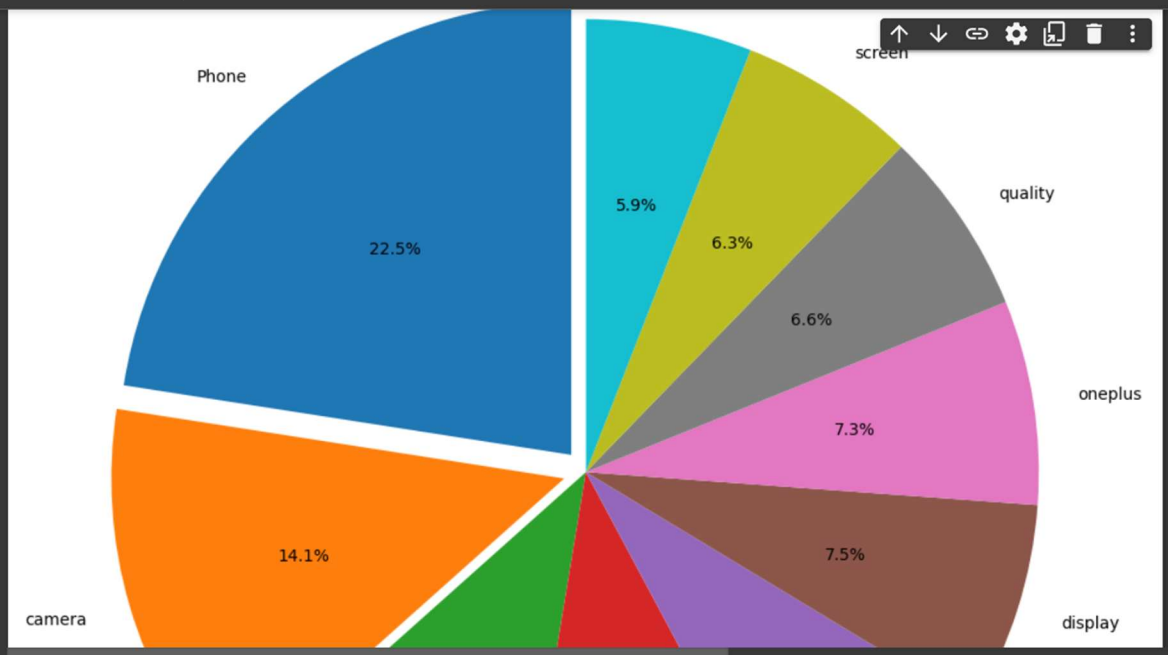
+ Code + Text Copy to Drive Reconnect

```python
#PIE CHART
#111_RahulBhujade
explode = (0.05,0.05,0,0,0,0,0,0,0,0)  # explode 1st and 2nd slice by 0.05 as compared to others
labels=['Phone',
        'camera',
        'one',
        'good',
        'battery',
        'display',
        'oneplus',
        'quality',
        'screen',
        'best',]

plt.pie(rslt['Frequency'], explode=explode,labels =labels , autopct='%1.1f%%', # autopct is a string us
        shadow=False, radius = 1, startangle=90) #shadow is used to create shadow of wedge
plt.legend( labels, loc='lower left',fontsize='x-small',markerfirst = True)
plt.tight_layout()
plt.title(' Commonly used words by Customers (Comments Column)')
plt.show()

import matplotlib as mpl
mpl.rcParams['font.size'] = 18.0     # matplotlib helps set runtime configuration parameters. here set
```

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

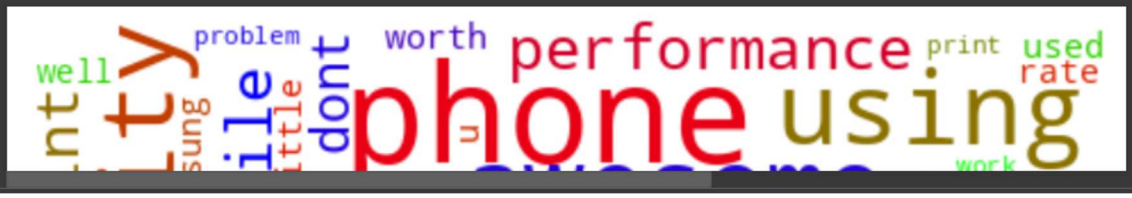


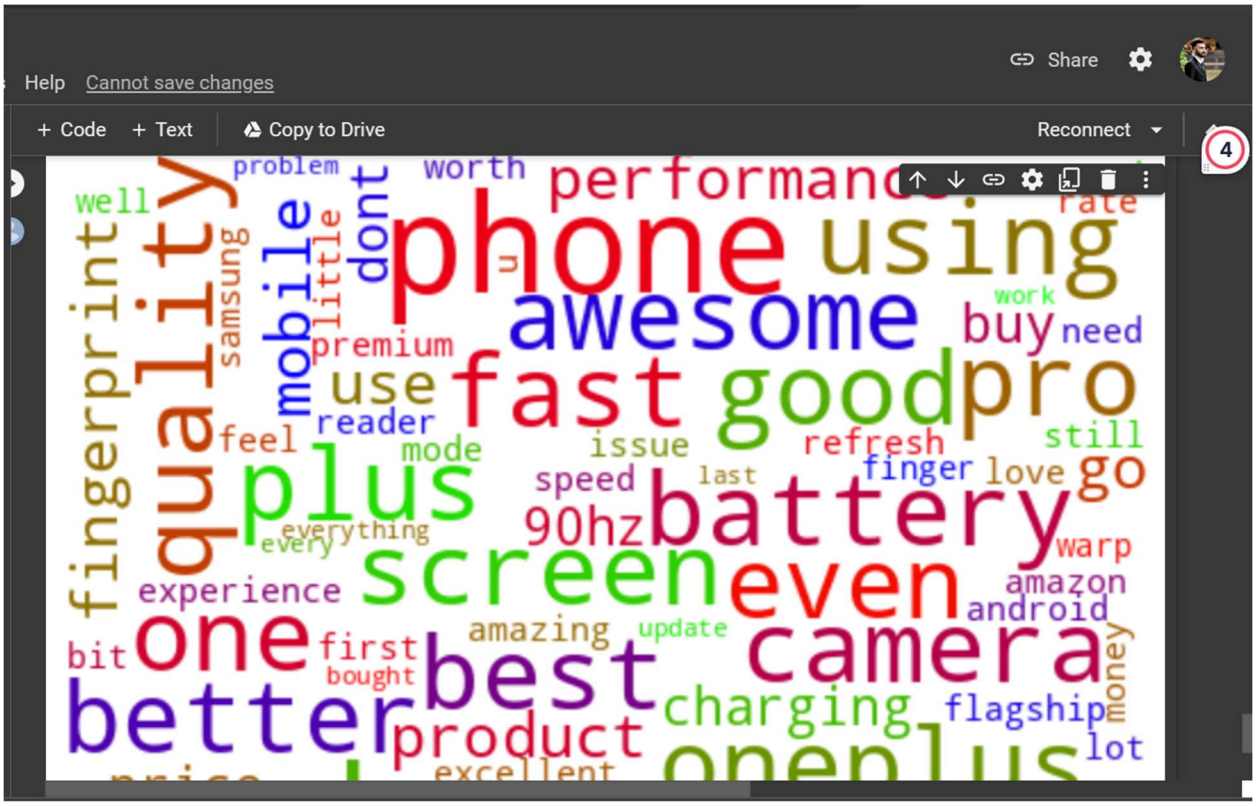

Help Cannot save changes

Share

+ Code + Text Copy to Drive Reconnect

```
#111_RahulBhujade # draw Word Cloud
import numpy as np
import pandas as pd
from os import path #This module contains some useful functions on pathnames. The path parameters are e
from PIL import Image #Instead of calling the Pillow module, we will call the PIL module as to make it
from wordcloud import WordCloud, STOPWORDS, ImageColorGenerator
import random

wordcloud = WordCloud(max_font_size=60, max_words=100, width=480, height=380,colormap="brg",
                      background_color="white").generate(' '.join(rslt_wordcloud['Word']))

plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.figure(figsize=[10,10])
plt.show()
```




Help
Cannot save changes
Share
+ Code
+ Text
Copy to Drive
Reconnect
4
problem
worth
performance
rate
well
fingerprint
quality
samsung
mobile
little
dont
phone
using
u
awesome
work
buy
need
premium
use
fast
good
pro
reader
feel
mode
issue
refresh
still
plus
speed
last
finger
love
go
everything
every
90hz
battery
warp
screen
even
amazon
experience
android
one
amazing
update
bit
first
bought
best
camera
money
better
product
charging
flagship
lot
excellent
oneplus